CFA INSTA
Institut National Supérieur des Technologies Avancées

python

Types de données



CFA-INSTA 2024

Python : types de données

Principaux types

Nom	Type	Description	Mutable
entiers	int	nombres : 3, 200, 5000	non
Nombres à virgule flottante flottante	float	2.3, 20.0	non
Chaines de caractères	str	’Hello”	non
Dictionnaire	dict	paires de valeurs : {‘’key”:”value”, ‘’name”:”Amy”}	oui
Tuples	tup	séquence ordonnée d’objets non mutables : (10, ‘’hello”, 20.4)	non
Ensembles	set	collection d’objets uniques et non ordonnées {‘’a”,”b”}	oui
Booléen	bool	valeur logique True ou False	non
Listes	list	séquence ordonnée d’objets [10,”hello”, 34.4]	oui

```
>>> type(3)
<class 'int'>
>>> type("hello")
<class 'str'>
>>> type({"key":"value", "key2":"value2"})
<class 'dict'>
>>> type([10.3, "hello"])
<class 'list'>
```

Python : types de données

Opérations sur les nombres

```
>>> int('42')
42
>>> float('42')
42.0
>>> str(42)
'42'
>>> import random
>>> random.randint(1,10)
10
>>> round(3.4)
3
>>> round(3.14159265358,2)
3.14
```

```
>>> from math import floor,ceil
>>> floor(3.14159265358)
3
>>> ceil(3.14159265358)
4
```

Python : types de données

Tuples

- Un tuple est un type de données de séquence
- Il peut contenir plusieurs valeurs dans une seule variable
- C'est un type ordonné : l'ordre des éléments est conservé
- Les valeurs ne sont pas uniques, elles peuvent être en double
- C'est indexé
- La longueur est variable
- Un tuple est immuable, il ne peut être modifié une fois défini
- Il est défini à l'aie des parenthèses
- Il est possible de convertir des tuples en listes ou en ensembles et inversement

```
>>> numbers = (1, 3, 4, 0, 4)
>>> numbers
(1, 3, 4, 0, 4)
>>> items = ("hello", True, False, 0.4, 34, 42)
>>> items[3]
0.4
>>> tuple([1, 3, 4])
(1, 3, 4)
>>> l1 = [1, 2, 3]
>>> l2 = [4, 5, 6]
>>> ( l1, l2)
([1, 2, 3], [4, 5, 6])
>>> ( *l1, *l2)
(1, 2, 3, 4, 5, 6)
```

Python : types de données

Listes

- La liste est une des structures de données les plus utilisées en python, avec les dictionnaires.
- Les éléments d'une liste peuvent être de n'importe quel type
- Il est facile d'accéder à un élément avec son index

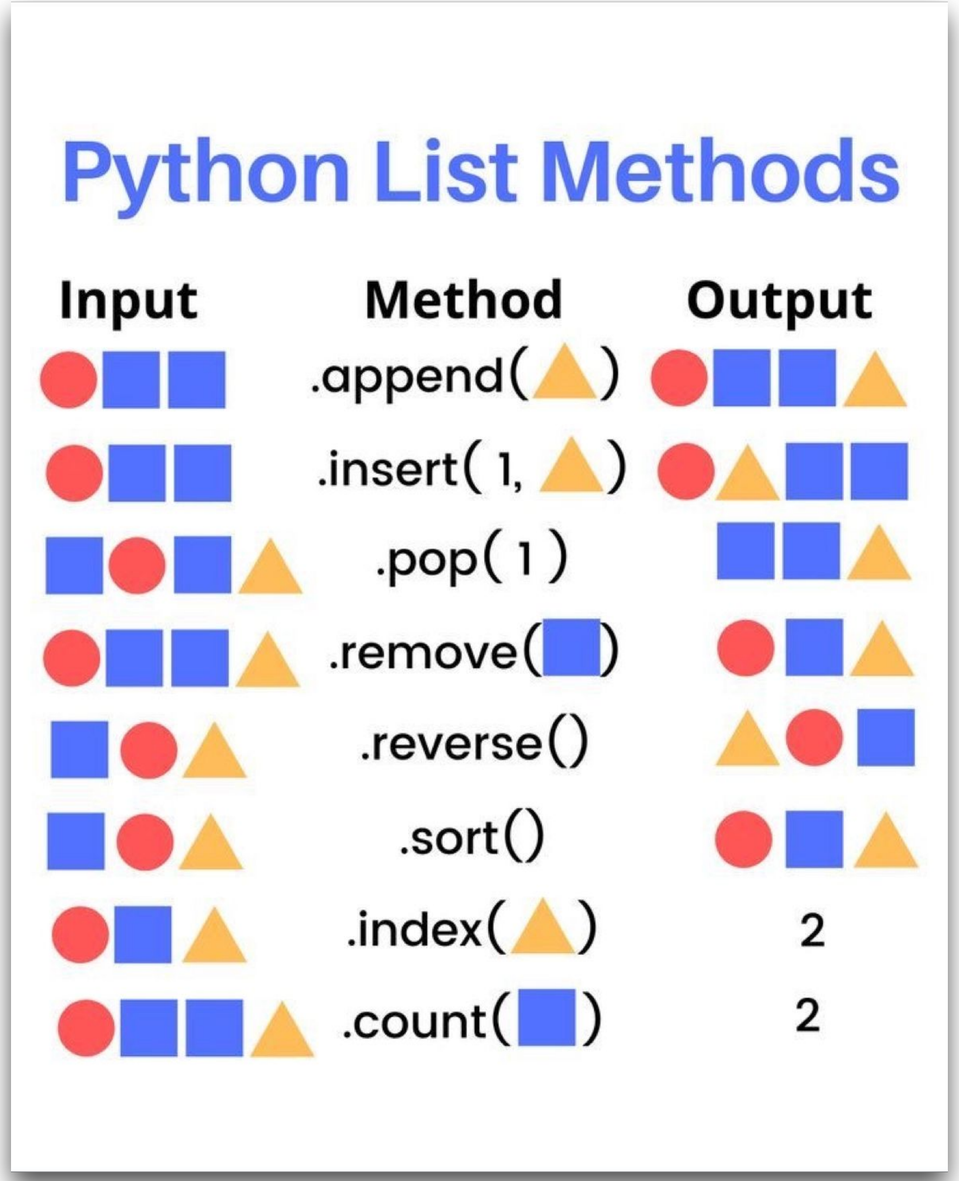
```
>>> liste1 = [1, 2, 3]
>>> listevide = []
>>> manytypeslist = [1, 3.4, "hello", {"key":"value"}]
>>> list(range(1,10))
[1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> manytypeslist[2]
'hello'
>>> manytypeslist[-1]
{'key': 'value'}
>>> biglist = [[1, 2], [3, 4], [5, 6]]
>>> biglist[1][1]
4
>>> manytypeslist.append("coucou")
>>> manytypeslist
[1, 3.4, 'hello', {'key': 'value'}, 'coucou']
>>> del manytypeslist[-2]
>>> manytypeslist
[1, 3.4, 'hello', 'coucou']
>>> manytypeslist.remove('hello')
>>> manytypeslist
[1, 3.4, 'coucou']
```

➡https://docs.python.org/3/tutorial/datastructures.html

Python : types de données

Listes

- La liste est une des structures de données les plus utilisées en python, avec les dictionnaires.
- Les éléments d'une liste peuvent être de n'importe quel type
- Il est facile d'accéder à un élément avec son index



Python : types de données

Ensemble

- Un ensemble est une collection d'élément distincts non ordonnés
- Les ensembles (set) peuvent servir à supprimer les redondances dans une liste

```
>>> names = {"Amy", "Gina", "Kevin"}
>>> names
{'Kevin', 'Amy', 'Gina'}
>>> names
{'Kevin', 'Amy', 'Gina'}
>>> names = {"Amy", "Gina", "Kevin"}
>>> names
{'Kevin', 'Amy', 'Gina'}
>>> names.add("Raymond")
>>> names
{'Kevin', 'Amy', 'Raymond', 'Gina'}
```

```
>>> names.update("Jake")
>>> names
{'e', 'a', 'Raymond', 'J', 'Gina', 'k', 'Kevin', 'Amy'}
```

```
>>> A = {1, 2, 3, 4, 5, 6}
>>> B = {4, 5, 6, 7, 8}
>>> print(A-B)
{1, 2, 3}
>>> print(B-A)
{8, 7}
>>> print(A^B)
{1, 2, 3, 7, 8}
>>> print(A&B)
{4, 5, 6}
>>> print(A|B)
{1, 2, 3, 4, 5, 6, 7, 8}
```

Python : types de données

Dictionnaires

- Les dictionnaires sont un des types de données les plus puissants
- Ils sont appelé « tableaux associatifs » dans les autres langages
- La syntaxe ressemble à du JSON
- On peut y ajouter tous les types de données

```
>>> phone_numbers = {"Amy":"6668253", "Gina":"1116668"}
>>> phone_numbers
{'Amy': '6668253', 'Gina': '1116668'}
>>> phone_numbers['Amy']
'6668253'
>>> del phone_numbers['Gina']
>>> phone_numbers
{'Amy': '6668253'}
>>> phone_numbers['Amy'] = '0000000'
>>> phone_numbers
{'Amy': '0000000'}
```

Python : types de données

Dictionnaires

- possibilité d'importer du json
- beaucoup d'opérations sont possibles

```
>>> import json
>>> jsondata='{"amy":"000999", "jake":"888999", "raymond":"777888", "gina":"111222"}'
>>> type(jsondata)
<class 'str'>
>>> json.loads(jsondata)
{'amy': '000999', 'jake': '888999', 'raymond': '777888', 'gina': '111222'}
>>> agent_dict = json.loads(jsondata)
>>> agent_dict
{'amy': '000999', 'jake': '888999', 'raymond': '777888', 'gina': '111222'}
>>> type(agent_dict)
<class 'dict'>
>>> 'gina' in agent_dict
True
>>> print(agent_dict.keys())
dict_keys(['amy', 'jake', 'raymond', 'gina'])
>>> print(agent_dict.values())
dict_values(['000999', '888999', '777888', '111222'])
```

Python : types de données

Dictionnaires

Méthode	Description	exemple
clear()	vider le dictionnaire	phone_numbers.clear()
get(key [, default_value])	Rechercher un idem avec une valeur par défaut	phone_numbers.get('Martha', 'Unknown person')
items()	retourner a liste des items	phone_numbers.items()
keys()	retourne la liste des clés	phone_numbers.keys()
values()	retourne la liste des valeurs	phone_numbers.values()
pop(key [, default_value])	retourne et supprime l'item	phone_numbers.pop('Martha')
popitem()	retourne et supprime le dernier élément ajouté	phone_numbers.popitem()
setdefault(key, value)	retourne la valeur de la clé, et si elle n'hésite pas, l'insérer	phone_numbers.setdefault('John Doe', 1234)
update(iterable)	ajouter les paires d'un autre étirable	dict1.update(dict2)